

11.- Backend (Practica 1)

Programación WEB I

Centro de Enseñanza Técnica Industrial

EMMANUEL BUENROSTRO 22300891 7F1

5 de Noviembre de 2025



Contents

1	Resumen	3
2	Desarrollo	3
2.1	Servidor y Cliente	3
2.2	Lenguajes de Programación	4
2.3	Base de Datos	4
2.4	Autenticación y Autorización	4
2.5	Seguridad	5
3	Bibliografía	5

Servidor y Cliente

El **cliente** es el dispositivo o navegador desde el cual una persona entra a una página web. Por otro lado, el **servidor** es una computadora que almacena los archivos, bases de datos y programas que permiten que el sitio funcione. Por ejemplo, cuando alguien entra a una red social, su navegador (cliente) envía una solicitud al servidor, y el servidor responde enviando los datos del perfil, publicaciones o mensajes.

Lenguajes de Programación

En el backend se usan lenguajes especializados para manejar la lógica y la comunicación con el servidor. Algunos de los más comunes son:

- **Python** (con frameworks como Django o Flask)
- **JavaScript** (con Node.js)
- **PHP**
- **Java**
- **C#**

Estos lenguajes permiten crear funciones como iniciar sesión, procesar pagos o guardar información en una base de datos. Por ejemplo, con Python y Django se puede hacer un sistema que guarde registros de usuarios y los muestre cuando alguien inicia sesión.

Base de Datos

Una **base de datos** es donde se guarda toda la información importante de una página, como usuarios, productos o publicaciones. Hay dos tipos principales:

- **SQL (estructuradas)** como MySQL o PostgreSQL.
- **NoSQL (no estructuradas)** como MongoDB.

El backend se conecta con la base de datos para guardar, actualizar o eliminar datos según lo que haga el usuario.

Autenticación y Autorización

La **autenticación** es el proceso de verificar que un usuario es quien dice ser, por ejemplo, cuando inicia sesión con su correo y contraseña. La **autorización** es el paso siguiente, donde se decide qué puede hacer ese usuario dentro del sistema. Por ejemplo, un administrador puede borrar publicaciones, pero un usuario normal no. Ambas son esenciales para mantener la seguridad de la aplicación.

Seguridad

La seguridad en el backend protege los datos de los usuarios y evita ataques. Algunas medidas comunes son:

- Cifrar las contraseñas antes de guardarlas en la base de datos.
- Usar conexiones seguras (HTTPS).
- Validar los datos que el usuario envía para evitar inyecciones de código.
- Mantener actualizado el servidor y las dependencias del sistema.

Un backend seguro evita que personas malintencionadas accedan a información privada o dañen el sitio.

§1 Resumen

El backend en programación web es la parte encargada de la lógica, el procesamiento y la conexión con la base de datos de una aplicación. A diferencia del frontend, que se centra en la interfaz del usuario, el backend opera en el servidor y responde a las peticiones del cliente. Utiliza lenguajes como JavaScript, Python o PHP para manejar la comunicación, la autenticación y la autorización de usuarios. Además, es responsable de mantener la seguridad mediante cifrado, validación y control de accesos. En conjunto, el backend es esencial para el funcionamiento estable, seguro y eficiente de cualquier sistema web.

§2 Desarrollo

§2.1 Servidor y Cliente

En la arquitectura web, el modelo **cliente-servidor** define la forma en que los datos y servicios son intercambiados. El **cliente** generalmente es un navegador web o aplicación móvil que solicita información o acciones, mientras que el **servidor** es una máquina remota que procesa esas peticiones y devuelve una respuesta.

El servidor ejecuta la lógica del negocio, gestiona la base de datos y controla el acceso a los recursos. Por ejemplo, cuando un usuario inicia sesión, el cliente envía una solicitud HTTP al servidor con los datos del formulario. El servidor verifica la información, accede a la base de datos y devuelve un resultado, como un mensaje de bienvenida o un error.

Los servidores pueden ser físicos o virtuales, y normalmente funcionan bajo sistemas operativos como Linux, Windows Server o macOS Server. Además, utilizan software como Apache, Nginx o Node.js para manejar las conexiones entrantes. La comunicación entre cliente y servidor se basa en protocolos como HTTP/HTTPS y formatos de datos como JSON o XML.

§2.2 Lenguajes de Programación

El backend se desarrolla utilizando lenguajes diseñados para procesar información, gestionar datos y comunicarse con el servidor. Algunos de los lenguajes más comunes incluyen:

- **JavaScript (Node.js)**: Permite crear servidores eficientes y escalables. Su modelo basado en eventos y asincronía lo hace ideal para aplicaciones en tiempo real, como chats o juegos en línea.
- **Python (Django, Flask)**: Lenguaje de alto nivel con sintaxis simple. Django ofrece una estructura completa con ORM, autenticación y panel administrativo, mientras que Flask es más ligero y flexible.
- **PHP (Laravel, Symfony)**: Popular en el desarrollo web tradicional. Laravel destaca por su facilidad para manejar rutas, controladores y bases de datos mediante Eloquent ORM.
- **Java (Spring Boot) y C# (ASP.NET)**: Se utilizan en entornos empresariales por su robustez, seguridad y compatibilidad con sistemas de gran escala.
- **Go y Rust**: Lenguajes modernos orientados al rendimiento, usados en servidores que requieren alta concurrencia o baja latencia.

Cada lenguaje se elige según los requerimientos del proyecto, el tamaño del equipo y la infraestructura disponible. En muchos casos, se complementan con frameworks que simplifican tareas repetitivas y mejoran la productividad.

§2.3 Base de Datos

Una parte esencial del backend es la **base de datos**, donde se almacena y organiza la información de la aplicación. Dependiendo del tipo de datos y su estructura, se puede optar por distintos modelos:

- **Bases de datos relacionales (SQL)**: Utilizan tablas con filas y columnas para almacenar la información. Son ideales para datos estructurados y relaciones complejas. Ejemplos: MySQL, PostgreSQL, SQL Server y SQLite.
- **Bases de datos no relacionales (NoSQL)**: Usan estructuras más flexibles, como documentos (MongoDB), pares clave-valor (Redis), grafos (Neo4j) o columnas (Cassandra). Son útiles para manejar grandes volúmenes de datos o aplicaciones distribuidas.

El backend interactúa con la base de datos mediante consultas, conocidas como *queries*, a través de SQL o mediante un ORM (Object-Relational Mapping), que traduce objetos del código en registros almacenados. La optimización de las consultas, el diseño de índices y la normalización de tablas son prácticas comunes para mejorar el rendimiento del sistema.

§2.4 Autenticación y Autorización

En el backend, la gestión de usuarios se basa en dos conceptos fundamentales: **autenticación** y **autorización**.

La **autenticación** verifica la identidad del usuario, usualmente mediante credenciales como nombre de usuario y contraseña, aunque también puede emplear métodos más avanzados como códigos OTP, autenticación biométrica o llaves de seguridad. Una vez autenticado, se genera una sesión o token que permite al usuario mantenerse identificado durante un periodo.

La **autorización**, por su parte, controla los permisos del usuario, es decir, qué recursos o acciones puede realizar. Por ejemplo, un usuario común no puede acceder al panel de administración. Para esto, el backend implementa sistemas de roles y políticas de acceso.

Actualmente, se usan tecnologías como:

- **JWT (JSON Web Tokens):** Tokens firmados digitalmente que almacenan información del usuario y su validez.
- **OAuth 2.0 y OpenID Connect:** Protocolos que permiten iniciar sesión mediante cuentas externas (Google, Facebook, GitHub, etc.).
- **Sessions y cookies:** Métodos tradicionales que guardan la autenticación en el servidor o navegador.

§2.5 Seguridad

La seguridad en el backend es un aspecto crítico, ya que una vulnerabilidad puede comprometer datos confidenciales o permitir ataques al sistema. Entre las principales medidas de seguridad se incluyen:

- **Uso de HTTPS:** Cifra la comunicación entre cliente y servidor para evitar el robo de información.
- **Cifrado de contraseñas:** Las contraseñas deben almacenarse utilizando algoritmos de hash como bcrypt, scrypt o Argon2.
- **Validación y sanitización de entradas:** Evita ataques de inyección SQL, XSS o ejecución de código malicioso.
- **Control de acceso:** Garantiza que solo usuarios autorizados puedan ejecutar ciertas acciones o acceder a recursos sensibles.
- **Gestión de errores:** Los mensajes de error no deben revelar información interna del sistema.
- **Actualización constante:** Mantener librerías, frameworks y dependencias al día reduce el riesgo de exploits conocidos.

Además, herramientas como *firewalls de aplicaciones web* (WAF), sistemas de detección de intrusos y pruebas de penetración ayudan a reforzar la seguridad de los servidores backend.

§3 Bibliografía

- A, D., A, D. (2023, 11 enero). ¿Qué es Bootstrap? - una guía para principiantes. Tutoriales Hostinger. <https://www.hostinger.com/mx/tutoriales/que-es-bootstrap>

- Arsys. (s. f.). Guía completa sobre Bootstrap. <https://www.arsys.es/blog/guia-completa-sobre-bootstrap>
- Calvo, L. (2024, 23 febrero). ¿Qué es Bootstrap? Guía para diseñadores y desarrolladores web. GoDaddy Resources - Spain. <https://www.godaddy.com/resources/es/crearweb/que-es-bootstrap>
- Carolina. (2025, 18 septiembre). Bootstrap: ¿qué es, para qué sirve y cómo instalarlo? Pingback. <https://pingback.com/es/resources/bootstrap>
- The Coder Cave, Programación y Tecnología. (2021, 30 julio). Aprende Bootstrap en 10 minutos [Vídeo]. YouTube. https://www.youtube.com/watch?v=XX1lX0A_9KQ